

Technical Disclosure Commons

Defensive Publications Series

11 May 2026

Remote Dispatch and Initiation of Client-Side Agentic Tasks via Cross-Device Control Protocol

Yijo Liu

Shadia Walsh

Lily Duraiswami

Yeun Su Shin

Tiffany Deonne Morris

See next page for additional authors

Follow this and additional works at: https://www.tdcommons.org/dpubs_series

Recommended Citation

Liu, Yijo; Walsh, Shadia; Duraiswami, Lily; Shin, Yeun Su; Morris, Tiffany Deonne; Borg, Carl Magnus; Krishna, Golden Gopal; and Chan, Clifford Tse Yan, "Remote Dispatch and Initiation of Client-Side Agentic Tasks via Cross-Device Control Protocol", Technical Disclosure Commons, (May 11, 2026)
https://www.tdcommons.org/dpubs_series/10076



This work is licensed under a [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/).

This Article is brought to you for free and open access by Technical Disclosure Commons. It has been accepted for inclusion in Defensive Publications Series by an authorized administrator of Technical Disclosure Commons.

Inventor(s)

Yijo Liu, Shadia Walsh, Lily Duraiswami, Yeun Su Shin, Tiffany Deonne Morris, Carl Magnus Borg, Golden Gopal Krishna, and Clifford Tse Yan Chan

Remote Dispatch and Initiation of Client-Side Agentic Tasks via Cross-Device Control Protocol

ABSTRACT

Architectures for agentic operations often involve a coupling of the input interface and the execution environment on a single piece of hardware. Such configurations can result in resource strain on mobile hardware, affecting battery longevity and data usage, and may encounter compatibility issues with desktop-centric web workflows. A dispatch protocol is disclosed that decouples the input interface from the execution environment. A dispatch payload is generated on a source device (e.g., a smartphone) or relayed via a third-party messaging channel (e.g., enterprise communication platforms or instant messaging applications). This payload includes a task prompt, browser configurations, and user context. The payload is transmitted to a target device identified through a device graph. A signal initiates a browser process and an agentic session on the target device, allowing the agent to utilize local hardware, stored authentication tokens, and local file systems to maintain data sovereignty. The protocol supports bidirectional communication, enabling real-time status updates and user "nudging" to resolve task bottlenecks. This method facilitates the asynchronous delegation of tasks within a user ecosystem based on hardware states or user-preferred communication channels.

KEYWORDS

agentic AI, autonomous, bidirectional task data gathering, browser configurations, channel-agnostic control, cross-device control protocol, local data persistence, decoupled execution, device graph, dispatch payload, fleet coordination, hardware resource delegation, local file system access, multiplayer tasking, persistent authentication tokens, remote dispatch, remote initiation, resource-intensive web tasks, task encapsulation, wake-and-execute logic

BACKGROUND

Current architectures for digital agents often involve a configuration where the input interface and the execution environment are located on the same hardware. This configuration can lead to significant resource consumption on portable devices, such as a smartphone, which may affect battery performance and data usage. Furthermore, web-based workflows designed for desktop environments can encounter compatibility issues when executed through mobile browsing interfaces.

Existing solutions for remote task execution, such as those described in some systems for voice-activated commands, often transmit a command to a personal computer to execute a task but do not adequately account for the specific browser configurations or the persistent authentication context used for autonomous web sessions. Other approaches that involve device selection based on battery states or basic task queuing lack a specific protocol for the asynchronous initiation of an agentic session in the background of a target device.

A technical challenge exists for a protocol that decouples the input interface from the execution environment. This would allow a task to be initiated on a first device while the actual browsing operations are performed by a second device, such as a workstation or laptop, using the local hardware resources, stored authentication tokens, and local file systems of the second device. This decoupling facilitates local data persistence, enabling the agentic session to access and process locally-stored sensitive information, such as tax documents, order receipts, or spreadsheets—that may not be synced to cloud environments or accessible via standard cloud-based AI services. Such a mechanism would facilitate the delegation of resource-intensive web tasks without involving immediate physical interaction with the target hardware.

DESCRIPTION

Overview / Solution Proposed

A method for delegating web-based operations between separate pieces of hardware within a common network or account environment is described. This protocol utilizes a source device as a controller and one or more target devices as execution nodes.

The disclosed technology provides a protocol for the delegation of web-based operations between separate hardware nodes within a common network or account environment. The described approach addresses the resource constraints and compatibility issues associated with performing intensive agentic tasks on portable hardware by decoupling the input interface from the execution environment. This is achieved through a remote dispatch mechanism where a source device, such as a smartphone, functions as a controller to initiate and manage tasks that are executed on a target device, such as a workstation. The core mechanism involves the generation of a structured data object that encapsulates the task parameters and user context, which is then transmitted to a remote execution node. This allows the target device to utilize its local processors, network interfaces, and stored authentication states to perform the task without immediate physical interaction from the user.

Task Encapsulation and Payload Generation

When a prompt is provided on a source device, such as a smartphone, a dispatch payload is generated. A dispatch payload is a structured data object that encapsulates the salient parameters for executing a task on a separate hardware node. This payload comprises a text-based instruction, such as a task prompt, along with specific browser configurations and user context. Examples of user context include persistent authentication tokens or cookies. The inclusion of these configurations allows the task to be processed on a remote machine, such as a desktop or

workstation, while maintaining the specific state and profile permissions of the user. This encapsulation facilitates the transfer of a complete execution environment from a controller interface to a remote execution environment without manual transcription of the original request. This payload includes a text-based instruction, specific browser configurations, and user context, such as persistent authentication tokens or cookies. The inclusion of these configurations allows the task to be processed on a remote machine while maintaining the user's specific state and profile permissions. The payload is transmitted over a secure, encrypted channel.

Target Identification and Selection

Available hardware linked to a user account is identified through a data structure representing a device graph. As used herein, a device graph is a data structure that, with user permission, maps and organizes hardware nodes associated with a specific user's account. It acts as the "inventory" for the remote dispatch system, allowing the controller (source device) to understand which "worker nodes" are available for delegation. A target device, such as a desktop computer or a laptop, is selected for task execution. In some instances, the selection is determined by evaluating the state of the available hardware, including current power levels or network throughput. For example, a task can be routed to a device with a wired network connection rather than one utilizing a metered wireless connection.

Asynchronous Execution and Background Initiation

The dispatch payload can be transmitted through a cloud-based messaging protocol, such as a push notification service. In one implementation, the control interface for the remote dispatch is channel-agnostic. While the source device may utilize a native browser UI, the protocol supports integration with third-party messaging platforms (e.g., enterprise chat services, instant messaging apps, or community communication tools) or voice-activated assistants as the primary command-

and-control conduit. In such configurations, a specialized bot within the third-party service acts as the bridge, relaying the dispatch payload to the target device and returning status updates or interactive prompts to the user within the third-party chat thread. Upon receipt of the payload, the target device initiates a browser process. This process can be started in a background state, allowing for the instantiation of an agentic session without involving immediate interaction with the primary display of the target device. The agentic session then processes the instructions within the local environment of the target device, utilizing local processors and network interfaces.

The protocol supports a bidirectional feedback loop between the execution node and the controller. Once the operation is finalized, a completion signal is transmitted back to the source device to update the task status. Additionally, the agentic session may transmit "heartbeat" updates or clarification requests to the source device if it encounters a decision point (e.g., selecting a specific product color). This allows the user to "nudge" the task through natural language or UI taps on the source device to ensure continuous execution without manual takeover.

Specific Implementation Examples

FIG. 1 illustrates a system 100 for the decoupled execution of web-based agentic tasks across separate hardware nodes. The system 100 generally comprises a source device 102, a

Cloud Messaging Service 110, and a target device 114.

FIG. 1: System Architecture Overview

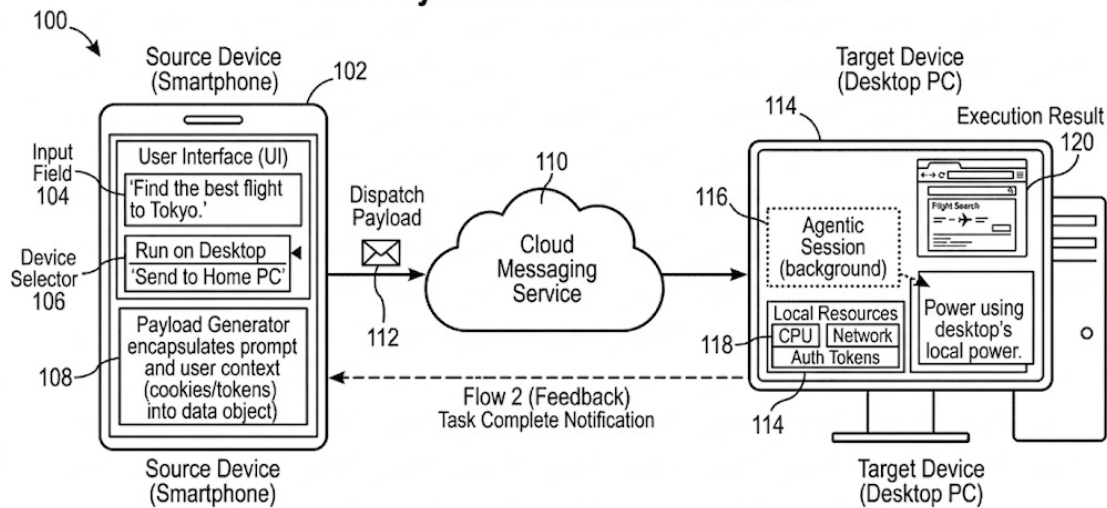


FIG. 1: System Architecture Overview

1. The source device (Controller)

The **source device** 102 (e.g., a smartphone) acts as the primary command-and-control interface for the user.

Input Field 104: Within a User Interface (UI), the user enters a natural language prompt or instruction. In the illustrated implementation, the prompt is "Find the best flight to Tokyo," representing a resource-intensive web task that would otherwise consume significant mobile battery and bandwidth.

Device Selector 106: The UI provides a mechanism for target selection, allowing the user to bypass local execution in favor of a remote node. This selection is supported by a device graph that identifies the hardware linked to the user's account (e.g., "Home PC" or "Work Laptop").

Payload Generator 108: This component performs Task Encapsulation. It generates a dispatch payload 112, a structured data object that captures the user's prompt alongside salient

user context, such as persistent authentication tokens, cookies, and browser settings (e.g., "Use my personal profile").

2. Transmission and Cloud Orchestration

Dispatch payload 112: The encapsulated task is transmitted as a lightweight signal from the source device 102.

Cloud messaging service 110: A cloud-based intermediary (e.g., a push notification service or messaging middleware) routes the payload to the designated target. This allows for Asynchronous Wake & Execute functionality, where the target device can be reached even if it is not currently in active use.

3. The Target Device (Execution Node)

The **target device 114** (e.g., a desktop PC) serves as the "worker node" that performs the actual computational labor.

Agentic session 116: Upon receiving the payload, the target device 114 instantiates a new agentic session. This process may be initiated in the background, allowing the browser to navigate and interact with web content without requiring immediate physical interaction or foreground display on the target hardware.

Local resources 118: The agent executes the workflow autonomously using the desktop's superior local resources, including the CPU, high-bandwidth network connection, and stored local cookies for seamless authentication.

Execution result 120: The final state of the task (e.g., an open tab with the flight results) is rendered locally on the target device 114.

4. Feedback Loop

Flow 2 (Feedback): Once the operation is finalized, the target device 114 transmits a Completion Notification back to the source device 102. This informs the user that the task, initiated remotely, has been successfully executed on the secondary hardware.

Accordingly, in the example of FIG. 1, a user enters a prompt on a source device 102 (e.g., a smartphone) to "Find the best flight to Tokyo and leave the tab open." The smartphone identifies a linked target device 114 via the device graph. A payload 112, including the prompt and the user's travel site cookies 108, is sent to the target device 114 (e.g., desktop PC). The desktop wakes its browser in the background, completes the search, and sends a confirmation to the source device 102 (e.g., a smartphone).

In another example, a task is queued on a mobile device at a specific time, such as "Download this report at 9 AM." The task is held in a queue and dispatched to a workstation when the scheduled time is reached, allowing the workstation to execute the download as the workday begins.

Advantages / Benefits

The disclosed technology provides several technical benefits over local execution models. Resource strain on portable hardware is reduced, as intensive computational and network tasks are delegated to devices with greater power and bandwidth capacities. Battery longevity and data conservation on mobile devices are improved. Furthermore, the use of a desktop-based execution environment resolves compatibility issues encountered by mobile user agents on some web workflows. The asynchronous nature of the protocol allows for the management of a fleet of devices from a single interface without the need for manual transcription or physical proximity to the execution hardware.

Potential Modifications or Scope

The protocol facilitates peer-to-peer dispatch between different users or groups. In this configuration, an agentic task is sent from a first user's device to a second user's device. The system further supports "multiplayer" task management, where a single agentic session on a target device can be monitored and influenced by multiple authorized controllers within a shared account or group (e.g., a family group coordinating a travel itinerary). Execution is initiated after a signal of approval is received from the recipient. Execution is initiated only after a signal of approval is received from the recipient. Additionally, the system supports automated load balancing, where the routing of a task is determined by identifying the device with the appropriate battery level or the more stable network connection among available nodes in the device graph.

Key Features

- Decoupling of the task input interface from the browser execution environment.
- Encapsulation of task prompts, browser configurations, and persistent authentication tokens into a transportable dispatch payload.
- Identification of execution nodes through a data structure representing a device graph.
- Asynchronous initiation of agentic sessions in a background state on a remote target device.
- Transmission of completion signals from the execution node back to the controller device.
- Delegation of tasks based on hardware state evaluations, such as network speed and power status.

CONCLUSION

The protocol for the remote dispatch and initiation of agentic tasks facilitates the decoupling of a task input interface from a browser execution environment. This decoupling is achieved through the generation of a dispatch payload on a source device, such as a smartphone, which is then transmitted to a target device, such as a workstation. This method addresses the resource constraints and compatibility issues associated with performing intensive web-based operations on portable hardware. By utilizing the local processors, network interfaces, and stored authentication tokens of the target device, tasks are executed in a background state without immediate interaction with the target hardware.